# S02.E2 - Ejercicio 02: Auditoría y Optimización Transaccional

*Sesión 02 | Módulo: Window Functions, Procedimientos Almacenados y PL/pgSQL*

**INFORMACIÓN Y PARÁMETROS DEL EJERCICIO**

**Entorno de Trabajo:** Base de datos *demodb* (Esquema *bookings*).
**Duración Máxima:** 25 Minutos.
**Modalidad:** Equipos de 2 a 3 estudiantes (Trabajo colaborativo distribuido en paralelo).
**Objetivo:** Implementar análisis analítico de tarifas de billetes mediante Window Functions y desarrollar automatizaciones procedurales con PL/pgSQL que soporten control transaccional explícito.

## Parte 1: Cuestiones Teóricas en Equipo (5 Minutos)

Discutan brevemente en equipo y respondan de forma sintética a las siguientes cuestiones antes de iniciar la programación:

**Cuestión 1.1 (Window Functions vs. GROUP BY):** Expliquen la diferencia conceptual entre clasificar billetes por rango de precio dentro de un vuelo mediante *NTILE(4) OVER (PARTITION BY flight_id ORDER BY amount DESC)* y agrupar precios mediante *GROUP BY*. ¿Por qué la Window Function permite conservar el número de billete (*ticket_no*) en cada fila mientras que *GROUP BY* colapsa la salida?

**Cuestión 1.2 (Procedimientos Almacenados y Transacciones):** En un procedimiento almacenado (*CREATE PROCEDURE*) que ajusta tarifas en lotes masivos, ¿por qué es posible y recomendable incluir órdenes *COMMIT;* intermedias? ¿Qué restricción técnica impide realizar esta misma operación dentro de una función convencional (*CREATE FUNCTION*)?

## Parte 2: Caso Práctico en Equipo - Trabajo en Paralelo (15 Minutos)

El departamento de auditoría y operaciones financieras de la aerolínea requiere procesar patrones de venta de billetes y actualizar tarifas inconsistentes. Para optimizar el tiempo de respuesta, distribuyan las tareas entre los integrantes del equipo:

### Integrante A (Rol: Analista de Ventanas - Window Functions)

Escriba una consulta *SELECT* sobre las tablas *bookings.ticket_flights* (alias *tf*) y *bookings.flights* (alias *fl*) que devuelva por cada billete y vuelo los siguientes campos:

- Identificador del vuelo (*fl.flight_id*) y número de billete (*tf.ticket_no*).
- Importe del billete (*tf.amount*) y clase de tarifa (*tf.fare_conditions*).
- Cuartil de precio del billete en su vuelo: *NTILE(4) OVER (PARTITION BY tf.flight_id ORDER BY tf.amount DESC) AS cuartil_precio*.
- Ingreso acumulado en el vuelo hasta la fila actual: *SUM(tf.amount) OVER (PARTITION BY tf.flight_id ORDER BY tf.ticket_no) AS ingresos_acumulados*.
- Filtre los resultados para mostrar únicamente los billetes pertenecientes al **cuartil 1** (tarifas más altas de cada vuelo) en vuelos pertenecientes al día de hoy o con más de 1,000 € acumulados.

### Integrante B (Rol: Desarrollador Procedural - PL/pgSQL Function)

Diseñe y cree la función procedural *bookings.fn_route_occupancy_stats(p_route_no varchar)* que retorne una tabla (*RETURNS TABLE*) con el resumen operativo de una ruta específica:

- **Parámetro de entrada:** Código de ruta (*p_route_no varchar*).
- **Columnas devueltas:** *flight_id integer*, *scheduled_departure timestamp*, *total_passengers bigint* y *route_avg_passengers numeric*.
- **Lógica interna:** Consulte los vuelos de la ruta combinando *bookings.flights* y *bookings.boarding_passes*.

- Calcule el promedio general de pasajeros por vuelo en toda la ruta utilizando una Window Function integrada: *AVG(COUNT(bp.seat_no)) OVER () AS route_avg_passengers*.
- Ordene el resultado por fecha de salida descendente.

### Integrante C (Rol: Ingeniero de Datos - PL/pgSQL Procedure & Audit)

Diseñe y cree el procedimiento almacenado *bookings.sp_audit_and_adjust_fare_rates(p_flight_id integer, p_min_amount numeric)*:

- **Parámetros:** Identificador de vuelo (*p_flight_id integer*) e importe mínimo de tarifa (*p_min_amount numeric*).
- **Lógica interna:** Identifique y actualice en *bookings.ticket_flights* todos los billetes del vuelo cuya tarifa sea inferior a *p_min_amount*, fijando su importe exacto a *p_min_amount*.
- Muestre un mensaje informativo mediante *RAISE NOTICE* indicando el número de registros modificados.
- Ejecute un *COMMIT;* explícito si se realizaron modificaciones para consolidar la transacción en disco de forma segura.

## Parte 3: Integración del Script Final del Equipo (5 Minutos)

Unan los componentes desarrollados por los tres integrantes en un único archivo ejecutable llamado *script_sesion02_ejercicio02.sql* siguiendo el orden lógico de ejecución:

1. Creación de la Función Procedural (Integrante B).
2. Creación del Procedimiento Almacenado de Auditoría (Integrante C).
3. Invocación de prueba del procedimiento: *CALL bookings.sp_audit_and_adjust_fare_rates(1234, 5000.00);*
4. Invocación de prueba de la función: *SELECT * FROM bookings.fn_route_occupancy_stats('PG0001');*
5. Ejecución de la consulta analítica con Window Functions (Integrante A).